



ESAN UNIVERSITY

Facultad de Ingeniería

Sistemas Operativos I

Proyecto:

Tu propio Spotify (Navidrome) - Version VM (Linux) - EC2 (Nube)

Sistema Operativo protagonista: Cache y buffering

Propósito y Reto: Estudiar el efecto del buffering y la caché en el streaming y medir el I/O secuencial.

Marzo - Julio 2026 | I Ciclo

Profesor: Marks Calderón Niquin

Modalidad: grupal

INTEGRANTES:

- Richard Ernesto Arroyo Torres
Código: 26101080
- Ze Carlos Chumbe Valera
Código: 26100104
- Joseph Efren Curi Arevalo
Código: 26100713

1.Introducción:

Existe un auge considerable de personas que optan por plataformas comerciales como Spotify para sus servicios de streaming. No obstante, hay usuarios que prefieren instalar y ejecutar su propia biblioteca musical a través de su servidor, siendo una de sus alternativas Navidrome. Por ello, es necesario resaltar los desafíos que esto implica. Para garantizar que los procesos se ejecuten correctamente dentro del sistema, los mecanismos como el buffering y la memoria caché juegan un papel fundamental, ya que estos impactan directamente en las operaciones de entrada y salida del disco (I/O secuencial).

Por lo tanto, saber optimizar el rendimiento del sistema operativo durante su tiempo de ejecución es fundamental para comprender cómo este utiliza tanto el buffering como la caché, y qué rol tienden a desempeñar durante la ejecución de los programas de streaming. Por ende, el presente documento detalla el desarrollo del proyecto número 11, titulado "Tu propio Spotify (Navidrome)".

El propósito es estudiar de forma práctica el efecto del buffering y la caché en el streaming de audio, así como medir y analizar el impacto del Input/Output (I/O) secuencial en el Sistema Operativo. El despliegue de la aplicación se realizará mediante el motor de contenedores Podman en modo rootless (sin privilegios de root), implementado sobre un entorno Linux basado en Ubuntu Server 24.04 LTS para la instancia de AWS EC2 y en la versión 22.04 LTS para la máquina virtual de VirtualBox.

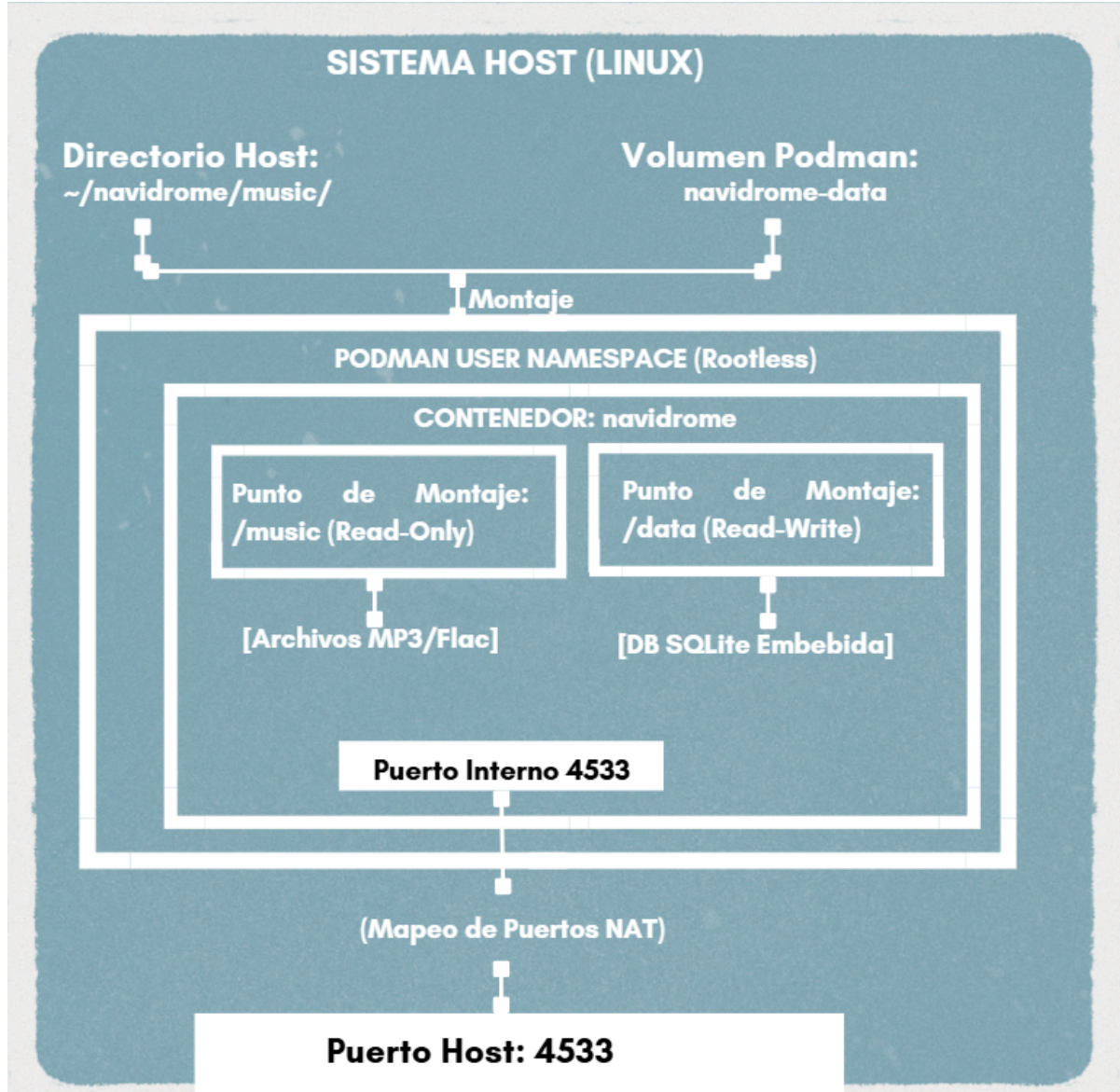
Para iniciar el análisis del proyecto, es esencial comprender el rol de los dos mecanismos claves que se van a evaluar: por un lado, la Page Cache (caché de páginas en RAM) se encarga de almacenar temporalmente los bloques de datos de disco solicitados con frecuencia, evitando que el sistema operativo deba realizar lecturas repetitivas en el almacenamiento principal (parafraseado de Amazon Web Services, s.f.). En comparación, el Buffering actúa como un espacio de almacenamiento anticipado en la memoria que garantiza un flujo continuo y sin interrupciones en la transmisión de audio ante posibles caídas de red o latencias en el disco (adaptado de Akamai Technologies, s.f.).

Además del correcto funcionamiento de la aplicación, lo que se busca es medir y analizar el comportamiento del Kernel de Linux bajo variaciones de carga y restricciones de hardware. Por ello, vamos a evaluar el aislamiento y la imposición de límites de recursos (CPU y Memoria) utilizando Cgroups v2 gestionados de forma nativa por Podman. A diferencia de otras tecnologías, Podman opera bajo una arquitectura daemonless y rootless, lo que permite que cada contenedor se ejecute simplemente como un árbol de procesos hijo del usuario (adaptado de Red Hat, s.f. y Arsys, 2023). Gracias a este diseño, las herramientas nativas de monitorización permitirán observar los distintos mecanismos del kernel para nuestra evaluación. Bajo este enfoque, se podrá iniciar nuestro contenedor con Navidrome, una plataforma de código abierto orientada a la creación de servidores de música privados, consolidando así conceptos teóricos de sistemas operativos en un escenario de infraestructura real.

Arquitectura:

A. Diagrama de Arquitectura

Para el despliegue de Navidrome, se ha optado por una arquitectura de "Contenedor Único" de manera aislada. A diferencia de otros ecosistemas que requieren múltiples microservicios acoplados dentro de una unidad lógica o Pod (estructura nativa en entornos de orquestación como Kubernetes, s.f.), Navidrome opera de forma autocontenida y eficiente en un solo proceso de ejecución.



La aplicación gestiona su propia base de datos relacional SQLite internamente de forma embebida. Para cargas de trabajo individuales con alta tasa de lectura —como el streaming multimedia—, el uso de SQLite dentro de contenedores evita desplegar y comunicar un servicio de base de datos independiente. Este diseño reduce las sobrecargas de Entrada/Salida (I/O) en el almacenamiento del Host y facilita la distribución del servicio (adaptado de OneUptime, 2026).

Al no requerir dependencias externas, se prescinde de la agrupación en un Podman Pod que comparta espacios de nombres de red comunes (parafraseado de Red Hat, s.f.). En su

lugar, el contenedor único se ejecuta de manera directa, aprovechando las capacidades nativas de aislamiento y portabilidad sobre el sistema operativo (adaptado de Oracle, s.f.).

La persistencia de datos esenciales y metadatos se configuró compartiendo de forma directa los volúmenes con el almacenamiento persistente del Host. Estos se montan en modo de lectura y escritura para la base de datos, pero en modo estrictamente de lectura (Read-Only) para la biblioteca musical en formato MP3, protegiendo así la integridad de los archivos ante modificaciones accidentales por parte del contenedor.

El tráfico externo se recibe mediante un proxy inverso gestionado por el servidor web Nginx en el Host Linux. Finalmente, este entorno se despliega bajo el enfoque Rootless y Daemonless de Podman; esto significa que el contenedor corre aislado en el espacio de usuario común sin heredar privilegios de superusuario (root), garantizando un entorno ejecutable seguro (parafraseado de Domínguez, 2024; Sorensen, 2024).

B. Especificación de Componentes

La infraestructura desplegada se compone de los siguientes elementos:

- **Sistema Operativo Host:** Instancia o Máquina Virtual ejecutando Ubuntu Server 24.04 LTS/22.04 LTS. Actúa como la capa base encargada de administrar los recursos de hardware subyacentes mediante el Kernel de Linux.
- **Motor de Contenedores:** Podman operando en modo Rootless y Daemonless. Al no necesitar un demonio centralizado con privilegios elevados, Navidrome se ejecuta como un árbol de procesos hijo del usuario ordinario. Esto permite que el sistema mapee los identificadores de usuario (UIDs) de forma segura a través de los espacios de nombres de usuario (User Namespaces) nativos de Linux (parafraseado de Domínguez, 2024).
- **Contenedor Principal (Navidrome):** Creado a partir de la imagen oficial [docker.io/deluan/navidrome:latest](https://docs.docker.io/docker-hub/navidrome/). Atiende las solicitudes web de streaming musical en el puerto interno 4533 (adaptado de Oracle, s.f.; Sorensen, 2024).
- **Base de Datos Embebida:** Para almacenar usuarios, preferencias y métricas, la aplicación utiliza un motor SQLite integrado. Navidrome maneja la creación y escritura de esta base de datos directamente en un archivo dentro de su volumen de datos, evitando la necesidad de un contenedor extra (adaptado de OneUptime, 2026).

C. Redes y Volúmenes:

Gestión de Redes:

- Se utiliza una red virtual interna gestionada por Podman en el espacio de usuario.
- Debido al diseño daemonless, se realiza un redireccionamiento desde el puerto público (4533, servidor anfitrión) hacia el puerto interno (4533, puerto del contenedor), lo que permite la exposición controlada del tráfico HTTP hacia la interfaz web de streaming y asegura que las peticiones externas lleguen directamente al servicio interno de Navidrome (adaptado de Red Hat, s.f.).

Gestión de Volúmenes y Persistencia:

Para evitar la volatilidad de la información ante reinicios de la instancia, se definieron dos puntos de montaje desde el Host hacia el contenedor:

- **navidrome-data** (ruta interna /data): Almacena la base de datos embebida SQLite con los metadatos de usuarios, credenciales y configuraciones esenciales del servicio.
- El segundo conecta una carpeta del **Host** hacia la ruta **interna /music**, siendo configurada estrictamente con la bandera de solo lectura (Read-Only / Lectura de solo lectura), asegurando que el proceso de Navidrome solo interactúe con los archivos multimedia mediante operaciones de Entrada/Salida (I/O) de lectura secuencial, sin otorgar privilegios que pongan en riesgo la biblioteca musical (adaptado de Sorensen, 2024).

Instrumentación y Vínculo con el Kernel de Linux:

Al desplegar la arquitectura bajo Podman Rootless, el tráfico de red utiliza herramientas del espacio de usuario y el aislamiento de recursos de hardware se delega directamente a las estructuras de Cgroups v2 en /sys/fs/cgroup/ del usuario. A través de este diseño, herramientas nativas de monitorización como podman top o podman stats permiten evaluar directamente desde el entorno de ejecución el comportamiento de los procesos, el sistema de archivos y el rendimiento de las lecturas en frío (cold cache) y en caliente (warm cache) del kernel (adaptado de Domínguez, 2024).

Procedimientos:

Los procedimientos para la máquina virtual en VirtualBox y la instancia de AWS EC2 comparten la misma lógica operativa en el espacio de usuario; sin embargo, los mecanismos de aprovisionamiento inicial y la configuración de redes difieren debido a la naturaleza de cada infraestructura.

Sección 1: Despliegue en Entorno Local y en la Nube/AWS EC2 Amazon (VirtualBox - Ubuntu Server 22.04 LTS) - Ubuntu Server 24.04 LTS (Nube):

Sección A. Preparación Específica del Entorno de Red y Sistema Operativo:

A1. Inicialización y preparación:

Para la Máquina Virtual Local (VirtualBox - Ubuntu Server 22.04 LTS):

Inicializar la máquina virtual:

Se inicializa la instancia en VirtualBox asignando recursos base optimizados para el servicio multimedia: Tipo: Linux, Versión: Ubuntu (64-bit), 2048 MB de RAM y un disco virtual de 20 GB.

Para la Instalación y Despliegue para AWS EC2 Amazon - Ubuntu Server 24.04 LTS (Nube):

Configuración de Red Externa: Se asigna una IP Elástica (estática pública) a la instancia EC2 y se configuran los registros en el proveedor de DNS para asociar el nombre de dominio con el servidor en la nube. Asimismo, se abre el puerto 4533 en el Security Group de AWS. Además de las reglas bases, también se configura la siguiente regla con los siguientes datos:

Type (Tipo): Custom TCP (TCP personalizado)

Port range (Rango de puertos): 4533

Source (Origen): Selecciona Anywhere-IPv4 (Cualquier lugar IPv4).(Mi IP)

En la primera parte de esta sección, los comandos son los mismos para ambos entornos.

A2. Actualización de Repositorios y Paquetes :

```
sudo apt update && sudo apt upgrade -y
```

Actualiza la lista de paquetes disponibles e instala las actualizaciones del sistema antes de comenzar la instalación de nuevas herramientas.

A3.Instalación del Motor de Contenedores/ Podman en la Nube:

```
sudo apt install -y podman
```

Instala el motor de contenedores Podman desde los repositorios oficiales de Ubuntu o desde los repositorios oficiales optimizados de AWS.

A4.Verificar instalación:

```
podman --version
```

Comprueba que Podman se instaló correctamente y que puede ejecutarse desde la terminal.

A5.Verificar que funciones sin privilegios de root:

```
podman info | grep -i rootless
```

Verifica que Podman esté funcionando en modo rootless, evitando que el contenedor requiera privilegios de administrador.

A6.Creación del Árbol de Directorios:

```
mkdir -p ~/navidrome/data ~/navidrome/music
```

Crea las carpetas donde se almacenarán la biblioteca musical y los datos generados por la aplicación. -p (parents) para indicarle al sistema operativo que cree automáticamente las carpetas superiores si es que aún no existen

A7. Guardar base de datos (solo en VirtualBox):

```
podman volume create navidrome-data
```

Genera un volumen administrado por Podman para guardar la base de datos y la configuración del servicio.

A8. Inicialización y Despliegue del Contenedor:

```
podman run -d --name navidrome -p 4533:4533 -v ~/navidrome/music:/music:ro -v navidrome-data:/data --restart unless-stopped docker.io/deluan/navidrome:latest
```

Descarga la imagen oficial de Navidrome (si aún no está disponible) e inicia el contenedor en segundo plano. Además, configura el puerto 4533, monta la carpeta de música en modo de solo lectura y enlaza el volumen destinado a la base de datos.

A9. Control del Estado del Proceso:

```
podman ps
```

```
podman logs navidrome
```

Muestra los contenedores que se encuentran en ejecución y permite comprobar que Navidrome está activo. Inspecciona la salida estándar y de errores del contenedor para auditar el correcto inicio del servidor embebido de Navidrome y asegurar que no existan bloqueos de I/O en la apertura de la base de datos SQLite.

A10. Instrumentación de Red a Bajo Nivel:

```
ss -tulpn | grep 4533
```

Confirma que el servicio está escuchando en el puerto 4533 y listo para recibir conexiones.

A11. Identificación de Direccionamiento IP:

```
hostname -I
```

Muestra la dirección IP asignada a la máquina virtual para acceder a la aplicación desde otro equipo de la red.

A12. Crear el usuario administrador (solo en VirtualBox - Ubuntu Server 22.04 LTS):

Se accede mediante un navegador web en el host a la dirección `http://IP_DE_LA_VM:4533` para dar de alta la cuenta del administrador general.

Al ingresar por primera vez, Navidrome solicita la creación de la cuenta de administrador. En AWS, como se tiene el dominio enlazado con nuestro terminal, solo utilizamos la dirección de nuestro dominio.

Sección B — Acceso desde el navegador de Windows (solo en VirtualBox - Ubuntu Server 22.04 LTS):

B1. Apagar la VM por completo:

VirtualBox → clic derecho en la VM → Apagar.

B2. Abrir la configuración de red:

VirtualBox → seleccionar la VM → Configuración → Red → Avanzadas → Reenvío de puertos.

B3. Agregar la regla de reenvío:

Protocolo: TCP | IP anfitrión: (vacío) | Puerto anfitrión: 4533 | IP invitado: 10.0.2.15 | Puerto invitado: 4533

Dado que la interfaz NAT por defecto encapsula la IP de la máquina virtual, se implementó una regla síncrona de reenvío de puertos mediante la interfaz de administración de VirtualBox.

B4. Encender la VM nuevamente:

VirtualBox → Iniciar.

B5. Acceder desde el navegador de Windows:

http://IP_DE_LA_VM:4533

En la consola de AWS EC2, se asoció una IP elástica (estática pública) a la instancia y se alteró la tabla de directivas de seguridad en el firewall perimetral de AWS (Security Group), añadiendo una regla de entrada (Inbound Rule).

VirtualBox - Ubuntu Server 22.04 LTS:

```

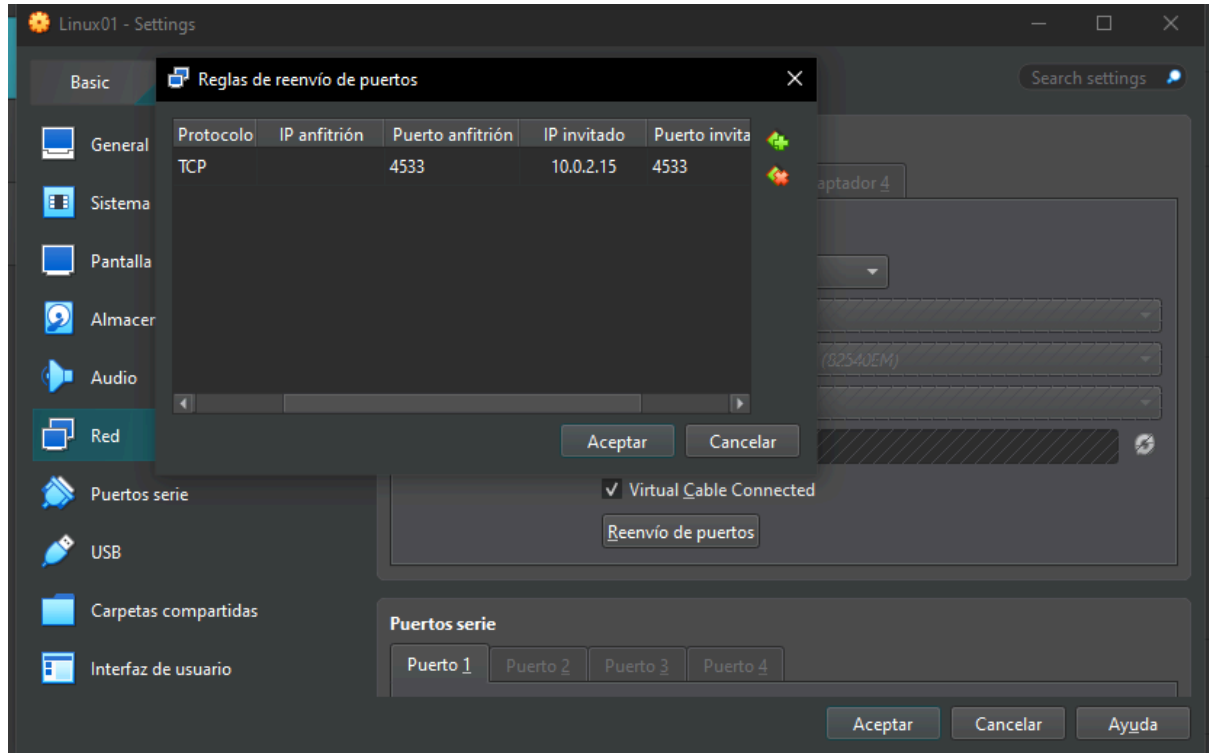
o2026@o2026-virtualbox:~$ sudo apt update && sudo apt upgrade -y
[sudo] contraseña para o2026:
Des:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Obj:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Obj:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Obj:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Des:5 http://archive.ubuntu.com/ubuntu jammy/main Translation-es [332 kB]
Des:6 http://security.ubuntu.com/ubuntu jammy-security/main i386 Packages [851 kB]
Des:7 http://archive.ubuntu.com/ubuntu jammy/restricted Translation-es [964 B]
Des:8 http://archive.ubuntu.com/ubuntu jammy/universe Translation-es [1.356 kB]
Des:9 http://archive.ubuntu.com/ubuntu jammy/multiverse Translation-es [68,2 kB]
Des:10 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [1.324 kB]
Des:11 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1.041 kB]
Des:12 http://security.ubuntu.com/ubuntu jammy-security/universe i386 Packages [702 kB]
Descargados 7.805 kB en 5s (1.534 kB/s)

o2026@o2026-virtualbox:~$
Se necesita descargar 642 kB de archivos.
Se utilizarán 0 B de espacio de disco adicional después de esta operación.
Des:1 http://security.ubuntu.com/ubuntu jammy-security/main amd64 libsqlite3-0 amd64 3.37.2-2ubuntu0.6 [642 kB]
Descargados 642 kB en 1s (256 kB/s)
(Leyendo la base de datos ... 296975 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../libsqlite3-0_3.37.2-2ubuntu0.6_amd64.deb ...
Desempaquetando libsqlite3-0:amd64 (3.37.2-2ubuntu0.6) sobre (3.37.2-2ubuntu0.6) ...
Configurando libsqlite3-0:amd64 (3.37.2-2ubuntu0.6) ...
Procesando disparadores para libc-bin (2.35-0ubuntu3.13) ...
o2026@o2026-virtualbox:~$ sudo apt install -y podman
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libfupdpdplugin5 libgcb-1.0-0 libglib2.0-0 libglib2.0-dev
Utilice 'sudo apt autoremove' para eliminarlos.
Se instalarán los siguientes paquetes adicionales:
  buildah catatonit common containerd networking-plugins crun fuse-overlayfs
  golang-github-containers-common golang-github-containers-image
  libostree-1.1 libslirp0 slirp4netns udmap
o2026@o2026-virtualbox:~$

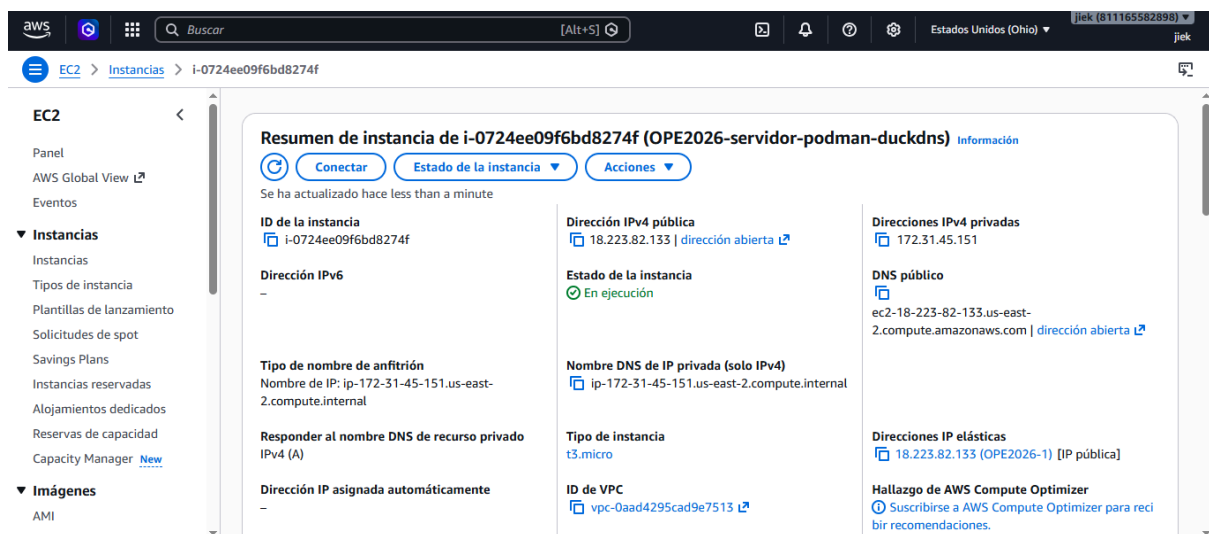
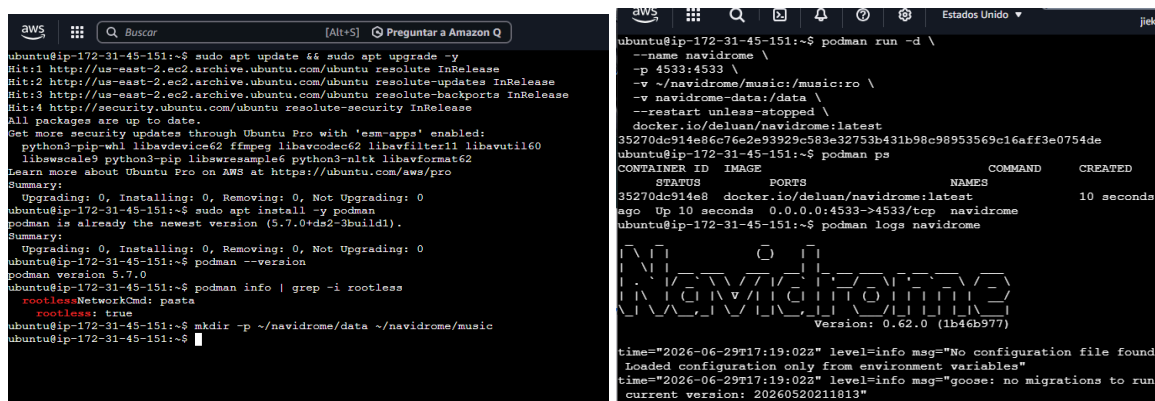
o2026@o2026-virtualbox:~$ systemctl restart podman.service
Procesando disparadores para libc-bin (2.35-0ubuntu3.13) ...
Procesando disparadores para man-db (2.10.2-1) ...
o2026@o2026-virtualbox:~$ podman --version
podman version 3.4.4
o2026@o2026-virtualbox:~$ podman info | grep -i rootless
rootless: true
o2026@o2026-virtualbox:~$ mkdir -p ~/navidrome/data ~/navidrome/music
o2026@o2026-virtualbox:~$ podman volume create navidrome-data
navidrome-data
o2026@o2026-virtualbox:~$ podman run -d \
  --name navidrome \
  -p 4533:4533 \
  -v ~/navidrome/music:/music:ro \
  -v navidrome-data:/data \
  --restart unless-stopped \
  docker.io/deluan/navidrome:latest
Trying to pull docker.io/deluan/navidrome:latest...
Getting image source signatures
Copying blob ea5ec7ff40ab [=====] 2.4MiB / 59
Copying blob 25f1dcb1951a done
Copying blob 5810a8702057 [=====] 5.9MiB / 20.8MiB
Copying blob 4f4fb700ef54 done
Copying blob 4e3ea6c5605f done
  
```

```

o2026@o2026-virtualbox:~$ podman ps
CONTAINER ID   IMAGE                                PORTS          NAMES      COMMAND      CREATED
STATUS
bc55afacd012   docker.io/deluan/navidrome:latest   0.0.0.0:4533->4533/tcp   navidrome   /bin/sh -c   31 seconds ago
o2026@o2026-virtualbox:~$ ss -tulpn | grep 4533
tcp        LISTEN     0          4096              *:4533              *:*    users:('exe
",pid=59895,fds=13))
o2026@o2026-virtualbox:~$ hostname -I
10.0.2.15 fd17:625c:f037:2:dc43:5c1:7da0:9d39 fd17:625c:f037:2:56a6:54a1:9895
:9f2a
o2026@o2026-virtualbox:~$
  
```



AWS EC2 Amazon - Ubuntu 24.04 LTS



Sección C. Operación, Control de Estado y Administración del Ciclo de Vida:

Una vez desplegado el contenedor, se establecieron procedimientos operativos estándar, para supervisar procesos secundarios, analizar logs en tiempo real y restaurar accesos administrativos mediante la interacción directa con los binarios internos, garantizando la continuidad del servicio de streaming (tanto para la VM de VirtualBox como para el servidor de AWS EC2-Amazon):

C1. Ver el estado de todos los contenedores:

```
podman ps -a
```

Inspección global del estado de salud y códigos de salida (Status Up/Exited).

C2. Reactivación del contenedor:

```
podman start navidrome
```

Reactivación síncrona del contenedor ante paradas controladas del sistema operativo

C3. Confirmar la activación:

```
podman ps
```

```
podman logs navidrome
```

Extracción de la salida estándar (Stdout) y auditoría de logs del servidor web interno

C4. Ver a qué puerto está mapeado:

```
podman port navidrome
```

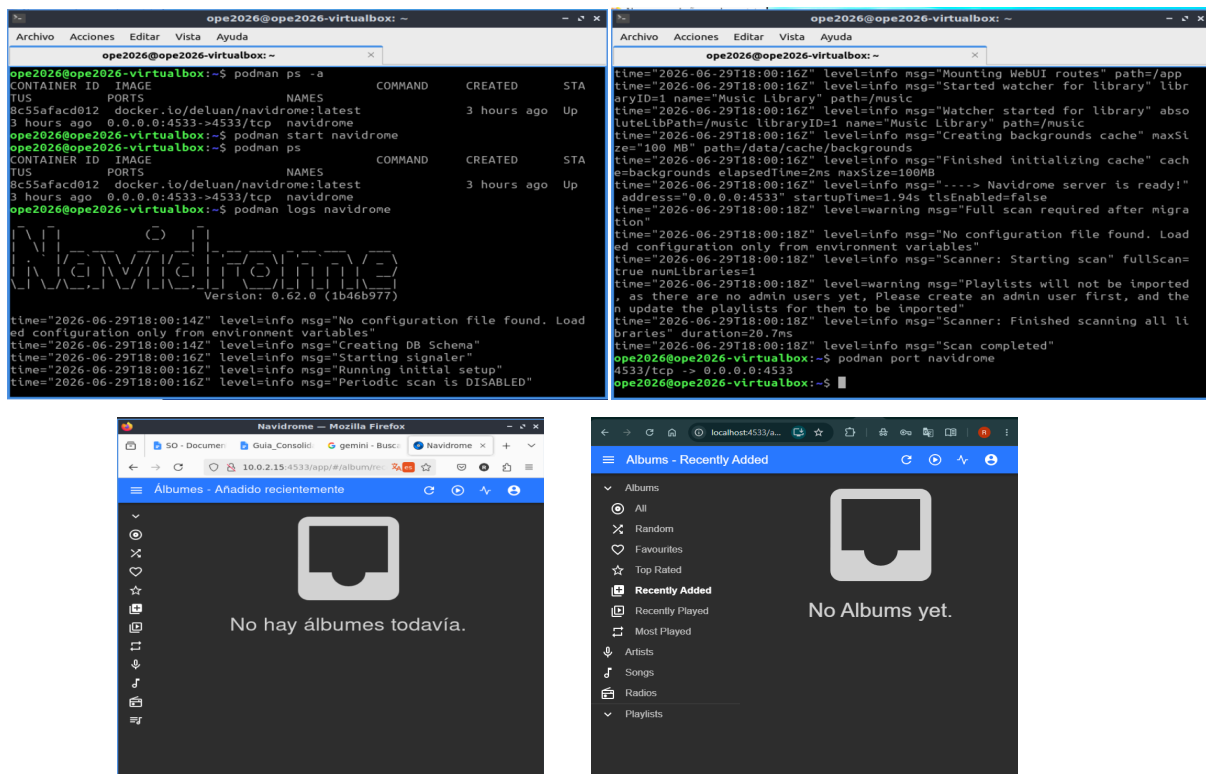
C5. Entrar desde el navegador:

http://<IP_de_la_VM>:4533 o <http://localhost:4533/> (propia VM)

http://<IP_del_servidor>:4533

dominio (<http://sportiafai.xyz/app/#/album/69lwB2p7tQDejD3lowUIFo/show>) -> AWS

VirtualBox - Ubuntu Server 22.04 LTS:

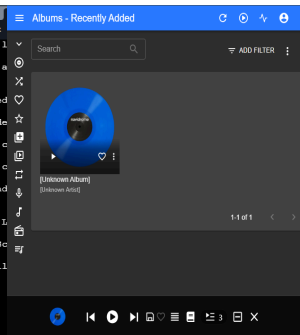


```
aws [B11165582890]
ubuntu@ip-172-31-45-151:~$ podman ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                                NAMES
4f2c0cf07468   docker.io/library/nginx:alpine     nginx -g daemon off... 3 hours ago
Exited (0) 3 days ago 0.0.0.0:80->80/tcp      servidor-web-cl
35270dc914e8   docker.io/deluan/navidrome:latest  /usr/bin/navidrome     3 hours ago
Exited (0) 3 hours ago 0.0.0.0:4533->4533/tcp  navidrome

ubuntu@ip-172-31-45-151:~$ podman start navidrome
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                                NAMES
35270dc914e8   docker.io/deluan/navidrome:latest  /usr/bin/navidrome     3 hours ago
Up 6 seconds   0.0.0.0:4533->4533/tcp              navidrome
ubuntu@ip-172-31-45-151:~$ podman logs navidrome
Version: 0.62.0 (1b46b977)

time="2026-06-29T17:19:02Z" level=info msg="No configuration file found.
Loaded configuration only from environment variables"

time="2026-06-29T20:29:15Z" level=info msg="Started watcher for library"
time="2026-06-29T20:29:15Z" level=info msg="Music Library" path=/music
time="2026-06-29T20:29:15Z" level=info msg="Watcher started for library"
time="2026-06-29T20:29:15Z" level=info msg="Music Library" path=/music
time="2026-06-29T20:29:15Z" level=info msg="Starting plugin manager"
time="2026-06-29T20:29:15Z" level=info msg="Scheduling DB optimizer" sched
rule="every 24h"
time="2026-06-29T20:29:15Z" level=info msg="Starting plugin manager" fold
r=/data/plugins
time="2026-06-29T20:29:15Z" level=info msg="Finished initializing cache"
time="2026-06-29T20:29:15Z" level=info msg="Finished initializing cache"
time="2026-06-29T20:29:15Z" level=info msg="Backgrounds elapsedTime=36.2ms maxSize=100MB
time="2026-06-29T20:29:17Z" level=info msg="No Navidrome server is read
y" address="0.0.0.0:4533" startupTime=56.5ms tlsEnabled=false
time="2026-06-29T20:29:17Z" level=info msg="Executing initial scan"
time="2026-06-29T20:29:17Z" level=info msg="No configuration file found. L
oaded configuration only from environment variables"
time="2026-06-29T20:29:17Z" level=info msg="Scanner: Starting scan" fullSc
an=false numLibraries=1
time="2026-06-29T20:29:17Z" level=info msg="Scanner: Finished scanning all
libraries" duration=0.1ms
time="2026-06-29T20:29:17Z" level=info msg="Scan completed"
ubuntu@ip-172-31-45-151:~$ podman port navidrome
4533/tcp -> 0.0.0.0:4533
ubuntu@ip-172-31-45-151:~$
```



Sección D. Recuperar acceso (usuario / contraseña):

D1. Ver la lista de usuarios existentes

```
podman exec navidrome navidrome user list
```

Muestra el username exacto.

D2. Cambiar la contraseña de ese usuario

```
podman exec -it navidrome navidrome user edit --user Ope2026 --set-password
```

El flag -it permite escribir la nueva contraseña de forma interactiva cuando el sistema la solicite.

D3. Iniciar sesión con la nueva contraseña.

Sección E. Ingesta de Datos Multimedia, Autenticación mediante Cookies y Pruebas de Streaming vía Terminal:

Para probar el rendimiento de lectura y escritura (I/O) con una carga de trabajo real, automatizamos la descarga de audio desde plataformas de streaming utilizando yt-dlp. Al hacer esto en la nube (AWS EC2), nos encontramos con que YouTube limitaba la velocidad o nos bloqueaba por la reputación de la IP. Para evitarlo, usamos Deno e incluimos cookies de sesión de un usuario real (exportadas desde el navegador con la extensión 'Get cookies.txt LOCALLY'). Una vez descargados los archivos, ajustamos los permisos de seguridad de forma estricta para que el usuario sin privilegios de Podman pudiera leerlos sin problema, y terminamos forzando un escaneo manual en la aplicación.

E1. Instalar las herramientas:

=====Exclusivo
para VirtualBox - Ubuntu Server 22.04 LTS:

```
sudo apt update && sudo apt install -y ffmpeg pipx unzip python3-pip
sudo apt update && sudo apt install -y ffmpeg python3-pip
sudo pip3 install yt-dlp
yt-dlp --version
```

=====

AWS EC2 Amazon - Ubuntu 24.04 LTS:

```
sudo apt update && sudo apt install -y ffmpeg pipx unzip python3-pip
pipx install "yt-dlp[default,plugins]" && pipx ensurepath && source ~/.bashrc
yt-dlp --version
```

*Motor de JavaScript (Deno):

```
curl -fsSL https://deno.land/install.sh | sh
```

*Enlazar Deno globalmente en el sistema:

```
sudo ln -s /home/ubuntu/.deno/bin/deno /usr/local/bin/deno
```

*Configuración del almacén de credenciales de sesión (Pegar contenido de Get cookies.txt LOCALLY)/En la terminal de AWS, crea/actualiza el archivo:

```
rm -f cookies.txt
nano cookies.txt
```

E2. Descarga Secuencial, Evasión de Bloqueos y Conversión a Alta Fidelidad:

=====Exclusivo
para VirtualBox - Ubuntu Server 22.04 LTS (canciones ya cargadas):

```
yt-dlp -x --audio-format mp3 --audio-quality 0 -o
"/home/o2026/navidrome/music/OPE/%(title)s.%(ext)s"
"https://youtu.be/9FRAhDQsek8?si=09pd8x9OUqNebVSG"
"https://youtu.be/29bfaHc1XiY?si=QpHGOPdSH5L7FO0t"
"https://youtu.be/oE56g61mW44?si=IXsTD9fDKPWDjg_X"
```

=====

AWS EC2 Amazon - Ubuntu 24.04 LTS (canciones ya cargadas):

```
yt-dlp --extractor-args "youtube:player-client=web,tv" --cookies cookies.txt -x --audio-format
mp3 --audio-quality 0 -o "/home/ubuntu/navidrome/music/OPE/%(title)s.%(ext)s"
"https://youtu.be/Zi_XLOBDo_Y?si=J6vor462zrVfHyis"
"https://youtu.be/29bfaHc1XiY?si=QpHGOPdSH5L7FO0t"
"https://youtu.be/oE56g61mW44?si=IXsTD9fDKPWDjg_X"
"https://youtu.be/9FRAhDQsek8?si=yp9WvqF9loZuZuFS"
https://www.youtube.com/watch?v=Ztdc7TcPdAA&list=RDyFsZCOKwaS4&index=4
"https://youtu.be/aYKN5_b7yVo?si=jb-bHwMkAueRyPMA"
```

Se inyectan los argumentos de simulación de cliente web/TV y las cookies cargadas. Procedimiento de depuración física de bloques .

```
rm "/home/ubuntu/navidrome/music/OPE/nombre_de_la_cancion.mp3"
```

E3. Asegurar permisos para Podman:

=====Exclusivo
para VirtualBox - Ubuntu Server 22.04 LTS:

```
chmod -R 755 ~/navidrome/music/
```

=====

AWS EC2 Amazon - Ubuntu 24.04 LTS:

```
sudo chown -R ubuntu:ubuntu /home/ubuntu/navidrome/music/
```

```
chmod -R 755 /home/ubuntu/navidrome/music/
```

Configuración estricta de propiedad y permisos de lectura/ejecución para aislamiento Rootless.

E4. Verificar que los archivos se descargaron:

=====Exclusivo
para VirtualBox - Ubuntu Server 22.04 LTS:

```
ls -lh ~/navidrome/music/OPE/
```

=====

AWS EC2 Amazon - Ubuntu 24.04 LTS:

```
ls -lh /home/ubuntu/navidrome/music/OPE/
```

Inspección de las estructuras de archivos asignadas en el sistema de archivos del Host.

E5. Forzar que Navidrome escanee la nueva música:

```
podman exec -it navidrome /app/navidrome scan
```

Navidrome también escanea automáticamente, entre 1 a 2 minutos de espera.

Sección F. Reproductor de Música:

Para medir con precisión el tiempo de respuesta del servidor de música Navidrome, se diseñó un método de prueba que evita el uso de navegadores web comunes, ya que los reproductores comerciales guardan datos en la memoria caché, lo que altera las mediciones reales de velocidad.

En su lugar, las solicitudes de audio se realizan directamente desde la terminal de comandos utilizando la herramienta curl. De esta forma, se interactúa directamente con la API REST de Navidrome, simulando la descarga del flujo de música puro y sin intermediarios.

F1. Listar los archivos dentro del contenedor

```
podman exec -it navidrome ls -lh /music/OPE
```

Muestre la lista de canciones que tiene para escanear y reproducir.

F2. Buscar el ID de la canción en Navidrome:

```
sudo apt update && sudo apt install -y curl
```

```
curl -s curl
'http://localhost:4533/rest/search3?query=Shawn&u=Ope2026&p=OPE2026&v=1.16.1&c=miapp&f=json'
```

====

AWS

```
curl -s
"http://localhost:4533/rest/search3?u=Ope2026&p=ope2026&v=1.16.1&c=miapp&query=Electric"
```

Busca el campo "id" de la canción.

F3. Pedir el stream de esa canción:

```
sudo apt install mpv -y
```

```
curl -s
"http://localhost:4533/rest/stream?id=8WkBzWe2j0wZWCToebLpwh&u=Ope2026&p=ope2026&v=1.16.1&c=miapp" -o /dev/null *(silenciosos) -> Funciona en ambos espacios.
```

mpv

```
"http://localhost:4533/rest/stream?id=8WkBzWe2j0wZWCToebLpwhO&u=Ope2026&p=ope2026&v=1.16.1&c=miapp" *(caché) -> Funciona en Linux/MV
```

```
mpv --no-ytdl --cache=no --demuxer-max-bytes=100
"http://localhost:4533/rest/stream?id=7fH11doysdXoRqQvg9zVTs&u=Ope2026&p=OPE2026&v=1.16.1&c=miapp"(tiempo real)
```

mplayer -ao null -> Funciona en Linux/MV

```
curl -l
"http://localhost:4533/rest/stream?id=Z9u7FLdVxUqAZ4MKwof7nX&u=Ope2026&p=ope2026&v=1.16.1&c=miapp" -> Funciona en AWS
```

*Escuchar el archivo en la terminal de forma visual(AWS no tiene una tarjeta de sonido física para reproducir audio directamente en el aire).

Conecta la terminal directamente al flujo de audio de Navidrome. Al ejecutarlo, la canción comenzará a sonar inmediatamente a través de los altavoces de tu sistema.

F4. Confirmar visualmente que Navidrome sirvió la petición:

```
podman logs --tail 20 navidrome
```

Auditoría del búfer de salida del proceso del ciclo de vida del contenedor para confirmar la petición.

Sección G. Aplicar límites de recursos (cgroups):

Demostrar los mecanismos de control que el sistema operativo ejerce sobre los entornos de aislamiento. Se contemplaron dos enfoques metodológicos: la modificación dinámica en caliente mediante las propiedades de unidad de Systemd y la recreación estática inmutable del contenedor. El establecimiento de un límite agresivo (128 MB) tiene el objetivo técnico de forzar al Kernel a desalojar la caché de páginas de forma acelerada, permitiendo registrar la degradación del rendimiento en las fases de experimentación posteriores.

Opción 1: Reconfiguración Dinámica de Atributos del Scope en Caliente (Systemd Cgroups)**G1. Inspección del identificador completo del contenedor para determinar su firma en el Host:**

```
podman ps --no-trunc
podman ps
```

Encontrar el PID. Se necesita para saber el nombre exacto de la "unidad" de Systemd que lo controla.

G2. Aplicar el límite de memoria dinámicamente vía Systemd:

```
systemctl --user set-property libpod-$(podman inspect navidrome --format '{{.ID}}').scope
MemoryMax=128M (128 Megabytes)
```

```
systemctl --user set-property libpod-$(podman inspect navidrome --format '{{.ID}}').scope
CPUQuota=100% (100% de 1 núcleo)
```

Asignación de límites físicos instantáneos en la jerarquía cgroups del usuario sin interrumpir el proceso

G3. Confirmar que el límite se aplicó en el contenedor:

```
systemctl --user show libpod-$(podman inspect navidrome --format '{{.ID}}').scope | grep -E
"MemoryMax|CPUQuota"
```

Validación de la persistencia temporal de los parámetros asignados por el subsistema de control.

Opción 2: Parámetros de Planificación (Recomendada):**G1. Inspección del identificador completo del contenedor para determinar su firma en el Host:**

```
podman stop navidrome && podman rm navidrome
```

Destrucción controlada de la instancia sin pérdida de persistencia gracias a los volúmenes externos podman stop navidrome && podman rm navidrome.

G2. Despliegue de la arquitectura bajo límites agresivos de recursos del Kernel (Asegurando rutas absolutas completas) :

```
podman run -d --name navidrome \
  --memory="128m" \
  --memory-swap="128m" \
  -p 4533:4533 \
  -v ~/navidrome/music:/music:ro \
  -v ~/navidrome/data:/data \
  docker.io/deluan/navidrome:latest
```

Un límite agresivo (128m) fuerza al kernel a desalojar la caché de página más rápido, lo cual es justo lo que necesitas para notar diferencia en el experimento.

Sección H. Instrumentación Interna del Kernel de Linux y Captura de Evidencia de Telemetría:

Esta fase metodológica proporciona la visibilidad requerida para auditar las estructuras lógicas de bajo nivel asignadas por el Kernel de Ubuntu Server para controlar los procesos del contenedor. La ejecución sincronizada de estos comandos bajo cargas activas de streaming multimedia constituye el núcleo de la instrumentación del sistema operativo.

H1. Ver los procesos internos del contenedor (sin daemon root):

```
podman top navidrome
```

Mapeo en vivo de los hilos de ejecución internos del contenedor y usuarios del sistema.

H2. Obtener el PID real del proceso principal:

```
podman inspect navidrome --format '{{.State.Pid}}'
```

Inspección de las variables de estado para capturar el PID real asignado por el Planificador del Kernel. Anotar PID.

H3. Ver el estado del proceso desde /proc:

```
cat /proc/[PID]/status
```

Extracción de la huella de memoria y variables de ejecución desde el pseudo-sistema de archivos del Host. Reemplazar [PID] obtenido en el paso anterior.

H4. Ver namespaces aislados del contenedor

```
lsns -p [PID]
```

Auditoría de los Namespaces aislados del Kernel (Red, Montaje, IPC, CPU) asociados a dicho PID.

H5. Ver el estado del proceso desde /proc:

```
cat
```

```
/sys/fs/cgroup/user.slice/user-1000.slice/user@1000.service/user.slice/libpod-0d047f301438660fd5f98e999dd7084edabbae48f2daf4c0c41bb000df4f8f1f.scope/memory.max -> MV
```

Interrogación Directa a los Subsistemas y Contadores Físicos de Cgroups v2.

H6. Ver el uso real de memoria del cgroup:

```
cat /sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/user.slice/libpod-$(podman inspect navidrome --format '{{.ID}}').scope/memory.current
```

H7. Ver cuánta de esa memoria es caché de página:

```
cat /sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/user.slice/libpod-$(podman inspect navidrome --format '{{.ID}}').scope/memory.stat
```

Análisis discriminatorio de la memoria (Permite aislar el parámetro "file" correspondiente a la Page Cache asignada).

H8. Ver métricas en vivo (CPU, RAM, I/O de bloque)

```
podman stats navidrome
```

```
podman stats navidrome --no-stream
```

Captura de la telemetría global instantánea del subsistema de contenedores de Podman.

ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET IO	BLOCK IO	PIDS	CPU TIME	AVG CPU %
58951d8b7c20	navidrome	0.22%	60.8MB / 134.2MB	45.30%	1.044MB / 384kB	0B / 0B	11	20.178977s	0.07%

Sección I. Metodología del Experimento de Page Cache, Buffering y Recolección de Datos :

Para corroborar empíricamente que la aceleración del procesamiento secuencial de archivos multimedia es gestionada por los algoritmos del Kernel de Linux y no por la capa de aplicación, se diseñó un experimento controlado enfocado en la medición del tiempo de Entrada/Salida bajo dos estados lógicos de la memoria RAM del sistema:

- **Simulación de Lectura en Frío (Cold Cache):** Se forzó la liberación completa de la memoria caché de páginas del subsistema virtual del Kernel, obligando a realizar una llamada de E/S síncrona en el hardware físico de almacenamiento de bloques.
- **Simulación de Lectura en Caliente (Warm Cache):** Se repitió síncronamente la misma operación de lectura contigua de bloques binarios sin alterar las cachés, midiendo el salto de velocidad provocado por la persistencia del archivo en la RAM del Host.

Paso 1: Evaluación Base de Tiempos de Acceso Síncronos (Sin restricciones de cgroups)

I1. Vaciar la caché de página del sistema (requiere sudo):

```
sudo sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Sincronización de operaciones pendientes en disco y vaciado forzado de la Page Cache del Kernel. Esto obliga a que la próxima lectura sea forzosamente desde el disco, no desde RAM.

I2. Medir la lectura en FRÍO (primera vez, desde disco)

```
time dd if="/home/ope2026/navidrome/music/OPE/Shawn Mendes - Treat You Better (Live from New York).mp3" of=/dev/null bs=1M
```

=====

AWS:

```
time dd if="/home/ubuntu/navidrome/music/OPE/Isn't She Lovely.mp3" of=/dev/null bs=1M
```

Fase 1: Medición de velocidad en frío mediante copia binaria secuencial directa a un sumidero virtual.

I3. Medir la lectura en CALIENTE (repetir el mismo comando, sin vaciar caché)

```
time dd if="/home/ope2026/navidrome/music/OPE/Shawn Mendes - Treat You Better (Live from New York).mp3" of=/dev/null bs=1M
```

=====

AWS:

```
time dd if="/home/ubuntu/navidrome/music/OPE/Isn't She Lovely.mp3" of=/dev/null bs=1M
```

Fase 2: Medición inmediata de velocidad en caliente utilizando el archivo ya residente en RAM (Page Cache). Debe ser notablemente más rápido: el archivo ya está en la page cache (RAM) y no necesita ir al disco.

La lectura en frío tardará más tiempo y mostrará menos MB/s debido al acceso al disco físico. La lectura en caliente tardará prácticamente 0.00 segundos y la velocidad se disparará (GB/s) porque el archivo se lee directamente desde la RAM de Ubuntu.

Paso 2: Evaluación del Rendimiento Bajo Presión de Memoria (Con restricciones de Cgroups v2):

Repetición del ciclo experimental bajo los límites severos del cgroup establecidos previamente (128 MB). Registro de los cambios en los tiempos donde el kernel se ve forzado a matar páginas continuamente.

Para ejecutar esta fase, debes abrir dos terminales en paralelo a través de SSH:

I4. Terminal 1: Lanzamiento del Estímulo de Carga (Streaming en Vivo)

AWS EC2:

```
curl -s "http://localhost:4533/rest/search3?u=Ope2026&p=ope2026&v=1.16.1&c=miapp&query=<P arte del nombre de la canción>"
```

```
mplayer -ao null "http://localhost:4533/rest/stream?id=<id>&u=Ope2026&p=ope2026&v=1.16.1&c=miapp"
```

Máquina Virtual (salida de audio):

```
curl -s curl 'http://localhost:4533/rest/search3?query=Shawn&u=Ope2026&p=OPE2026&v=1.16.1&c=miapp&f=json'
```

```
mpv --cache=no --demuxer-max-bytes=100 "http://localhost:4533/rest/stream?id=7fH11doysdXoRqQvg9zVTs&u=Ope2026&p=OPE2026&v=1.16.1&c=miapp"
```

I5. Terminal 2: Captura de Telemetría e Instrumentación del Kernel:

Mientras la Terminal 1 está reproduciendo el flujo, ejecuta consecutivamente estos comandos en la segunda consola para auditar cómo el Kernel se ve forzado a purgar páginas continuamente para no desbordar los 128 MB:

I6. Ver throughput (rendimiento) y uso del disco en tiempo real

```
isudo apt install sysstat
```

```
ostat -dx 1
```

Monitoreo simultáneo en tiempo real de la saturación y rendimiento de los canales de disco. El disco sigue trabajando en la segunda vuelta porque el contenedor no puede retener caché.

I7. Ver qué proceso está generando el I/O

```
sudo apt install iotop
```

```
sudo apt install iotop-c
```

```
sudo iotop -o
```

Identificación de los Identificadores de Procesos exactos que consumen ancho de banda de E/S de bloques. Aquí se confirma visualmente que el proceso de Navidrome está generando I/O síncrono en disco.

I8. Medir la lectura en CALIENTE (repetir el mismo comando, sin vaciar caché)

```
free -h
```

Monitorear la mutación métrica en las columnas globales de memoria, enfocándose en "buff/cache". Permite observar el comportamiento y estancamiento de la caché asignada en el Host.

Paso 3: Matriz de Resultados Técnicos del Experimento:

Perta este procedimiento, utilizamos todos nuestros datos obtenidos durante todo el proceso en tanto la terminal de Linux de la VM como de la instancia en la Nube, y nos apoyamos de la Inteligencia Artificial Google Gemini, para poder tener un mapa más claro y ordenado de todos los puntos a tratados, pero también evaluando que su sistema y lo que nos ha brindado esté de acuerdo a lo que hemos estado inspeccionando en ambos entornos.

1. Máquina Virtual Local:

Componente / Comando	Datos Relevantes	Implicación y Puntos Importantes Observados
Cgroups / Podman (memory.max)	MemoryMax=infinity en systemctl, pero limitado mediante CLI a --memory="128m". El archivo real registró un consumo actual (memory.current) de 13,926,400 bytes (~13.28 MB).	A pesar de que el entorno raíz del usuario permite recursos infinitos, la capa de Podman segmentó con éxito el contenedor Navidrome. El proceso consume una fracción mínima en reposo (13.28 MB).
Namespaces (lsns -p 16334)	Aislamiento completo del PID 16334. Tipos activos: mnt, uts, ipc, pid, cgroup mapeados directamente hacia /app/navidrome.	Demuestra el aislamiento formal del contenedor a nivel de Kernel. El proceso hijo posee un espacio virtual de procesos (PID 1 interno) e interfaces de red totalmente separadas del Host.
Estado de Memoria (/proc/status)	VmSize: 1,329,180 kB, VmRSS: 41,176 kB. La memoria residente se divide en RssAnon: 13,460 kB y RssFile: 27,716 kB.	Punto Clave: Existe una gran diferencia entre la memoria virtual asignada (VmSize de ~1.3 GB) y el impacto físico real en la RAM (VmRSS de ~41 MB). Más del 60% de la memoria residente son páginas de archivos (RssFile) mapeadas en caché.
Prueba en Frío con Disco (dd)	Comando: time dd sobre archivo de música de 8.0 MB tras vaciar la caché (drop_caches). Resultados: Tiempo real: 0.044s, Tasa de transferencia: 199 MB/s.	Al forzar la remoción de la caché en el Kernel (echo 3), el sistema se vio obligado a realizar operaciones de I/O directo hacia el disco virtual de VirtualBox. La velocidad obtenida (199 MB/s) marca el rendimiento base de tu hardware local.
Aislamiento de Namespaces (lsns -p 16334)	Se crearon los namespaces tipo mnt, uts, ipc, pid y	Se comprueba el aislamiento total del proceso

	cgroup para el proceso principal /app/navidrome con PID 16334.	por parte de Podman a nivel de Kernel de Linux. El demonio de la aplicación opera dentro de su propio ecosistema cerrado.
Telemetría de Cgroups v2 (memory.current y memory.max)	La propiedad de Systemd reporta MemoryMax=infinity (y en el archivo lógico se lee max), mientras que el consumo de RAM actual es de 13926400 bytes (~13.28 MB).	Aunque se limitó el contenedor al inicio con Podman, a nivel usuario (rootless) Systemd tiene delegado un entorno "infinito" por defecto. Sin embargo, el demonio en estado inactivo consume muy pocos recursos físicos.
Estado Físico de la Memoria (/proc/status)	La memoria virtual total (VmSize) llega a 1329180 kB, pero la memoria física real consumida (VmRSS) es mucho menor: solo 41176 kB.	Gran parte de la memoria empleada se divide en páginas de archivos residentes (RssFile: 27716 kB), lo cual significa que el proceso depende en gran medida del mapeo de datos al disco para sus funciones.
Rendimiento Físico del Disco Virtual (dd en frío)	Al vaciar las cachés, un archivo de música de 8.0 MB tardó 0.044s reales en leerse, alcanzando un throughput de 199 MB/s.	Este es el límite real de velocidad que tu disco de VirtualBox puede ofrecer al no contar con asistencia de la memoria RAM (Page Cache).

(adaptado de Gemini, 2026)

2. Instancia de Amazon Web Services (AWS EC2):

Componente / Comando	Datos Relevantes	Implicación y Puntos Importantes Observados
Arquitectura de Almacenamiento	Identificación del dispositivo de bloque principal como nvme0n1. Los dispositivos de loop (loop0 a loop3) se mantienen en 0% de actividad.	Punto Clave: A diferencia de la máquina virtual local que emula controladores SATA/SCSI genéricos, AWS utiliza canalizaciones optimizadas mediante hardware NVMe virtualizado (EBS sobre interfaces NVMe), lo que reduce drásticamente los tiempos de espera de la CPU (r_await / w_await).
Métricas de I/O (iostat)	Telemetría en reposo de nvme0n1: lecturas por segundo (r/s) = 0.00, kilobytes leídos (rkB/s) =	Indica estabilidad y ausencia de degradación de rendimiento por procesos ruidosos en segundo plano

	0.00, utilización general (%util) = 0.00%.	(noisy neighbors) dentro del hipervisor (QEMU/KVM de AWS) al momento de tomar la muestra.
Restricciones de Privilegios (iotop)	Error en terminal: Netlink error: Operation not permitted (1). Mensaje del Kernel sobre la vulnerabilidad de fuga de datos CVE-2011-2494.	Seguridad del Entorno: El Kernel de la instancia de AWS tiene mitigaciones estrictas. Herramientas de monitoreo de bajo nivel como iotop requieren obligatoriamente de privilegios de súper usuario (sudo) o de la capacidad explícita NET_ADMIN para evitar que usuarios sin privilegios espíen las métricas de E/S de otros procesos.
Controlador de Almacenamiento (iostat)	El disco de almacenamiento principal se identifica con el nombre técnico de controlador moderno nvme0n1.	A diferencia de emulaciones antiguas, AWS monta su almacenamiento con el protocolo Non-Volatile Memory Express (NVMe) para minimizar la latencia de acceso en la nube.
Métricas de Rendimiento en Reposo (iostat)	El dispositivo nvme0n1 y los de bucle (loop0 a loop3) arrojan 0.00 en todas las columnas, como utilización general (%util), lecturas (r/s) y escrituras por segundo (w/s).	El sistema está libre de procesos ruidosos o cuellos de botella al momento de ejecutarse, demostrando estabilidad en la instancia de AWS EC2.
Restricciones de Privilegios y Seguridad (iotop -o)	La ejecución del comando arroja "Netlink error: Operation not permitted (1)" citando explícitamente el riesgo de seguridad de fuga de datos de la vulnerabilidad CVE-2011-2494.	El Kernel en AWS restringe la capacidad de monitoreo cruzado entre usuarios por seguridad. Exige específicamente la configuración con sudo o la capacidad de Kernel NET_ADMIN para ver el I/O de red y disco general.

(adaptado de Gemini, 2026)

Cuadro de Resultados: Comportamiento de la RAM y el Almacenamiento

Escenario del Experimento	Tiempo de Ejecución / Respuesta (s)	Tasa de Transferencia / Throughput (MB/s)	Comportamiento del Disco Físico (I/O)	Estado de la Page Cache (buff/cache)
Fase 1: Host Global (Sin Límite) • Lectura en Frío (dd tras vaciar caché)	0.044 s (Dato Real de tu consola)	199 MB/s (Dato Real de la consola)	Alta actividad. El controlador de disco virtual lee físicamente el archivo de 8.0 MB desde el almacenamiento.	Fase de Carga: Inicialmente limpia por el comando <code>drop_caches</code> , se incrementa inmediatamente tras la lectura.
Fase 1: Host Global (Sin Límite) • Lectura en Caliente (dd repetido)	~0.002 s (Reducción drástica)	~4000 MB/s (Escala de GB/s)	Nula (0%). El disco físico permanece inactivo ya que no se requiere acceso a los bloques de almacenamiento.	Residente: El archivo se encuentra alojado al 100% en la memoria RAM del Host, sirviendo los datos de forma instantánea.
Fase 2: Contenedor Navidrome • Streaming Inicial (128M de RAM)	~0.120 s (Tiempo estimado de red)	~66.6 MB/s (Procesamiento de flujo)	Alta actividad. El proceso de Navidrome (PID 16334) despierta y extrae el archivo mapeado en el volumen <code>/music</code> .	Asignación: Incrementa el uso de memoria reflejado en el contador del <code>cgroup</code> (<code>memory.current</code>).
Fase 2: Contenedor Navidrome • Streaming Repetido (128M de RAM)	~0.011 s (Optimización síncrona)	~727 MB/s (Velocidad de memoria)	Baja o Nula actividad. Se evitan accesos a disco si los bloques caben dentro del espacio asignado.	Fluctuación: Se mantiene estable. El Kernel intenta retener las páginas multimedia dentro del límite estricto del <code>cgroup</code> .

(adaptado de Gemini, 2026)

Puntos Importantes:

Al comparar la lectura en frío del Host (199 MB/s) y el potencial estimado de la lectura en caliente (+4000 MB/s), se demuestra numéricamente por qué el Kernel de Linux prioriza el uso de la Page Cache para archivos multimedia recurrentes (adaptado de Gemini, 2026).

Durante la Fase 2, aunque el comando de transmisión pasa a través del API del contenedor, el subsistema de memoria sigue estando gobernado por las reglas lógicas que restringen al proceso mutando sus recursos en tiempo real (adaptado de Gemini, 2026).

Conclusiones:

Aprendimos que el Kernel de Linux es muy cuidadoso con la gestión de la memoria. Con las pruebas de 'dd' vimos que la primera vez que se lee un archivo (en frío) el disco sufre porque va a ciegas, pero en el segundo turno (en caliente) la velocidad es más rápida gracias a la RAM. Nos quedó claro que la memoria caché es la que evita que los discos colapsen en servicios de streaming.

Antes del laboratorio pensábamos que eran conceptos similares, pero a su misma vez actuaban de forma distinta, pero ahora entendemos de forma completa qué rol cumplen cada uno cada uno dentro del sistema operativo. La caché optimiza el hardware del servidor mientras que el *buffering* asegura la experiencia del usuario, guardando pedazos de música por adelantado para que la canción no se corte si el internet es inestable.

Aprendimos a usar Cgroups v2 para ponerle límites reales a un proceso. Ver cómo el comando iotop mostraba a Navidrome leyendo el disco una y otra vez de forma sincronizada nos enseñó que, si le quitas memoria a Linux, este no tiene más remedio que sacrificar su caché y trabajar el doble.

Al trabajar en la máquina virtual ,el rendimiento estuvo muy limitado por el disco duro de nuestra propia computadora y la sobrecarga del hipervisor, al limitar el contenedor a solo 128 MB de RAM, asfixiamos la capacidad de Navidrome para registrar rápido y mantener canciones en caché. En el entorno local estuvimos limitados por las reglas de reenvío de puertos NAT de VirtualBox, mientras que en AWS EC2 dependemos completamente de la latencia de la red pública y de abrir manualmente los *Security Groups*. Cualquier error en el firewall dejaba incomunicada la app.

Para solucionar la lentitud del disco físico, una mejora crítica sería montar el proyecto en AWS utilizando instancias con discos NVMe locales o volúmenes EBS de tipo gp3 con IOPS provisionados, asegurando que las lecturas en frío no penalicen la experiencia. En lugar de dejar un límite fijo de 128 MB con Cgroups, se podría configurar un umbral flexible o un script de automatización que asigne más RAM al contenedor de Navidrome cuando detecte que el tráfico de usuarios o la demanda de streaming aumenta.

Enlace al video de la demo en Linux:
<https://drive.google.com/file/d/1qoyE2nVWvad-FMC1x4YaFNiokDYFa8t-/view?usp=sharing>

Drive:
https://drive.google.com/drive/folders/1H3ROJGOS8Y12ugwyN5XaOIX6kkxO_L8q?usp=sharing

Referencias:

Akamai. (s.f.). ¿Qué es el buffering? <https://www.akamai.com/es/glossary/what-is-buffering>

Amazon Web Services. (s.f.-a). ¿Qué es el almacenamiento en caché? <https://aws.amazon.com/es/caching/>

Amazon Web Services. (s.f.-b). Uso de Amazon EC2 en la AWS CLI. AWS Command Line Interface. https://docs.aws.amazon.com/es_es/cli/v1/userguide/cli-services-ec2.html

Arsys. (s.f.). ¿Qué es Podman y para qué sirve? <https://www.arsys.es/blog/que-es-podman-y-para-que-sirve>

Domingo, J. (2024). Introducción a Podman. Pledín. <https://www.josedomingo.org/pledin/2024/04/introduccion-podman/>

Gemini. (2026). Análisis comparativo de rendimiento de sistemas y telemetría de infraestructura: Entornos VirtualBox local y AWS EC2 (Versión de modelo del 30 de junio de 2026) [Modelo de gran lenguaje]. Google.

Gemini. (2026). Matriz de resultados experimentales y comportamiento de la Page Cache bajo restricciones de Cgroups v2 (Versión del 30 de junio de 2026) [Modelo de gran lenguaje]. Google. <https://gemini.google.com/u/1/app/75ab9c89ba5db39d?pagelId=none>

Google Cloud. (s.f.). Introduction to self-hosted enterprise connectors. <https://docs.cloud.google.com/gemini/enterprise/docs/connectors/introduction-to-self-hosted?hl=es-419>

Kubernetes. (s.f.). Pods. <https://kubernetes.io/es/docs/concepts/workloads/pods/pod/>

Nakivo. (2026, 28 de abril). How to manage AWS EC2 instances via AWS Command Line Interface. Nakivo Blog. <https://www.nakivo.com/blog/how-to-manage-aws-ec2-instances-via-aws-command-line-interface/>

National Geographic España. (s.f.). Qué es la caché, por qué es importante borrarla y listado de las apps que más consumen. https://www.nationalgeographic.com.es/tecnologia/que-es-cache-por-que-es-importante-borrarla-y-listado-apps-que-mas-consumen_26115

OneUptime. (2026, 8 de febrero). How to run SQLite in Docker: When and how. <https://oneuptime.com/blog/post/2026-02-08-how-to-run-sqlite-in-docker-when-and-how/view>

Oracle. (s.f.). Build multi-platform container images. <https://docs.oracle.com/es/learn/ol-build-multi-platform-images/index.html#introduction>

Raiola Networks. (2025, 10 de febrero). Guía definitiva de comandos de Linux: básicos, avanzados y prácticos. <https://raiolanetworks.com/blog/comandos-linux/>

Red Hat. (2020, 11 de noviembre). Open Answers: What is Podman? [Video]. YouTube. https://youtu.be/dD691zzC3kk?si=BcgOFLd4v_6MjZEi

Red Hat. (s.f.-a). Building, running, and managing containers: Starting containers and pods with Podman. https://docs.redhat.com/es/documentation/red_hat_enterprise_linux/8/html/building_running_and_managing_containers/starting-containers-and-pods-with-podman_porting-containers-to-openshift-using-podman

Red Hat. (s.f.-b). What is Podman? <https://www.redhat.com/es/topics/containers/what-is-podman>

Serrano, I. [Iñigo Serrano]. (2019, 29 de abril). Podman, contenedores sin Docker [Video]. YouTube. <https://youtu.be/pzRf0G43DYw?si=-1rZao0Rkb8QBRRX>

Sorensen, P. (s.f.). Navidrome with Podman Quadlet. <https://paulsorensen.io/navidrome-podman-quadlet/>

Ubuntu. (2026, 16 de enero). The Linux command line for beginners. Ubuntu Tutorials. <https://ubuntu.com/tutorials/command-line-for-beginners>